

EngageIQ: Smart Engagement Opportunity Scorer

Technical Brief | Big Data (BAX 423) Final Project

Author: Saurav Kanegaonkar | **Student ID:** 924465543

Live application: `engageiq-148810228333.us-central1.run.app` (Google Cloud Run, public, no login needed)

Submitted materials: `code/` (single-command FastAPI app), `data/` (offline 20,995-record SQLite snapshot), `brief.pdf`, `prompts.md`, `README.md`.

1. System Architecture

EngageIQ reframes the problem as an attention-portfolio recommender: it treats a unit of a professional's time as the item being allocated, and ranks opportunities by expected impact per unit of effort against a weekly budget. The system is one linear pipeline, ingestion to deduplication to embedding to scoring to multi-stage ranking to adaptive learning, with a parallel batch-analytics path for trends and a downloadable brief at the end. It is meant to be usable by a non-data-scientist, so the engine is wrapped in a product, not a notebook.

Five source adapters (GitHub REST, Reddit, Hacker News, Dev.to, Bluesky) each normalize to a single canonical **Opportunity** schema. Incoming records pass a two-layer deduplicator: a from-scratch Bloom filter (Lecture 2) as a fast probabilistic pre-check, backed by a database primary key as the certain guarantee. The offline snapshot holds 20,995 deduplicated, 2026-only records spanning all 15 required technical domains.

Dev.to	6,607	Reddit	3,768
Bluesky	5,482	GitHub	1,000
Hacker News	4,138	Total	20,995

Two SQLite databases are kept deliberately separate: a read-only corpus (`engageiq.sqlite`, the snapshot a grader runs with no API keys) and a read-write user-state database (`engage.sqlite`: accounts, profiles, per-card interactions, an append-only event stream, and the learner weights), so the snapshot stays clean while real usage persists.

A FastAPI backend loads the engine once at startup and serves both the single-page product UI and a JSON API from one process. The whole thing is one container on Google Cloud Run, with CPU PyTorch and the two transformer models baked into the image so cold starts never re-download them; it scales to zero when idle. The product flow is onboarding, then a weekly Plan screen where the user steers a Contribute / Learn / Discuss / News split, then the ranked Hub feed, a Trends tab, and the exportable brief.

The Hub presents a ranked feed where each card carries a plain-language “Why this?” explanation (the actual signal drivers, in words), an LLM-drafted Suggested Action (a concrete post, comment, or pull request to open) with a copy button, and engage / skip / save controls that feed the learner live. The Trends tab visualizes the batch analytics: a from-scratch force-directed topic mind map, domain momentum (share change week over week), fresh-this-week inflow, and the most active communities

(hyperlinked out). The weekly brief exports as a server-rendered PDF: an academic-style document with the user’s plan, the top moves grouped by activity bucket, the trends in their space, and what the learner has inferred about them.

2. BAX-423 Techniques and Why

The pipeline integrates four course technique families. Each was chosen against a cheaper baseline and benchmarked.

Opportunity text and the user’s interest profile are encoded with Sentence-BERT (`all-MiniLM-L6-v2`, 384-d), chosen over `bge-small` by an evidence-based comparison (cleaner domain separation, simpler pipeline). Retrieval is exact cosine over an L2-normalized FAISS `IndexFlatIP`. This is the matching core, and it is robust to noisy domain labels: retrieval precision@10 is 0.89. Embeddings were chosen over keyword search because the corpus is short, jargon-dense Reddit topics and GitHub issues where semantic context matters.

A candidate-generation to scoring to re-ranking funnel: stratified per-source FAISS recall, hard filters (platform, domain, language-exclusion), a cross-encoder relevance re-rank (Stage-2), a goal-weighted impact score, MMR for diversity, and a final time-budget knapsack that fills the weekly hour budget (“This Week’s Plan”). Scoring is goal-conditioned: four engagement modes (Contribute, Discuss, Monitor, Promote) set the signal weights and the suggested-action verb. The cross-encoder was added because, as the benchmark below shows, dense embeddings alone lose to keyword search on this jargon corpus; reading query and document jointly recovers the gap.

The batch-analytics layer computes trends over the full corpus with a from-scratch Count-Min Sketch (term frequencies, never-undercount guarantee) and HyperLogLog (distinct contributors, 16 KB). On the real streams they hit 1.16% max error at 9.8x less memory (CMS) and 0.43% error at 48.7x smaller (HLL) versus exact counts. Crucially both merge without rescanning, which is what lets the dashboard roll a persona’s domains up on the fly.

A per-(user, persona) online weight-learner (Widrow-Hoff over the nine scoring signals, warm-started from the mode defaults and renormalized so it stays interpretable) updates on every engage / skip / save and persists, so learning accumulates across reloads.

3. Matching and Ranking Benchmarks

The Stage-2 re-ranker was benchmarked with an LLM-as-judge harness (relevance 0 to 3, pooled and cached for reproducibility), reporting nDCG@10 and P@10.

Stage-2 matching strategy	nDCG@10
Dense (MiniLM cosine, alone)	0.63
TF-IDF (keyword)	0.77
Hybrid (dense + keyword)	0.70
Cross-encoder re-rank (chosen)	0.76

The honest finding: dense embeddings alone (0.63) underperform plain TF-IDF (0.77) on this short, jargon-heavy corpus. Rather than hide it, the design treats it as the reason for the cross-encoder Stage-2 (0.76), which matches keyword quality and wins specifically for the skill-match personas (Sofia, David). The full assembled pipeline (recall, filters, cross-encoder, impact, MMR, knapsack) reaches a mean nDCG@10 of 0.711 across the four test personas. Adaptive learning was validated separately over 60 simulated feedback rounds against a hidden preference: nDCG@5 rose from 0.679 (mode defaults) to 0.960 (learned), with the weights converging to the hidden preference. A 53-check automated

suite covers the sketch accuracy and merge property, the analytics schema, and the trends and brief endpoints; all pass.

4. Test Persona Results

Each of the four provided personas was evaluated against its pass criteria and each of the six core capabilities. All twelve persona pass-criteria pass; the capability coverage is below.

Core capability	Sofia	David	Lina	Raj
1. Ingestion + dedup + storage	Pass	Pass	Pass	Pass
2. Embedding + similarity retrieval	Pass	Pass	Pass	Pass
3. Scoring + multi-stage ranking	Pass	Pass	Pass	Pass
4. Adaptive learning from feedback	Pass	Pass	Pass	Pass
5. Batch analytics + trend detection	Pass	Pass	Pass	Pass
6. Dashboard + Why-this + brief	Pass	Pass	Pass	Pass

The persona-specific criteria are met by the goal-conditioned modes. Sofia (Contribute) gets beginner-friendly GitHub issues with C++/Rust excluded and a contribution-worthiness gate that favors well-run, well-specified repos; David (Discuss) gets Kubernetes and infra threads, not general web dev, and a standout-repo lane (high activity, few contributors); Lina (Monitor) gets a velocity radar ranked by recency and star/comment surge with week-over-week trend analytics; Raj (Promote) gets developer-tools discussion threads, and the learner deprioritizes low-engagement threads after simulated skips.

5. Limitations

Snapshot recency bias. The offline scrape skews to recent weeks (a collection artifact), so a raw volume-over-time line would mislead. Trend signals therefore use each domain’s share of weekly activity on complete weeks only, which is robust to the bias, and the limitation is stated in-product.

Domain-label precision. Labels began as keyword matches (about 62% precision). A multi-label re-classification with an independent verification gate lifted this to 78%, but several niche domains (for example Python data engineering, AI research) remain weaker, so a small fraction of items carry an imperfect domain tag.

GitHub supply. GitHub is the smallest source (1,000 records), so a Contribute-heavy persona can see fewer GitHub items than its plan requests even though the matching is correct; the candidates exist in the corpus but do not always rank into the top slice. This is a data-supply limit, not a ranking bug.

Hosting trade-offs. The free Cloud Run instance has an ephemeral filesystem, so runtime user accounts and learned weights reset on a redeploy or a cold start; the read-only corpus and all ranking are unaffected. Durable accounts would need a managed database, which was out of scope for a free single-instance deployment.

6. AI-Assisted Development

Development was AI-assisted throughout (Claude), used for code generation, multi-agent data-labeling and summarization workflows, and copy. The full prompt log is in `prompts.md`; one representative example: *“Re-classify these GitHub issues and Reddit posts into the 15 technical domains with a primary plus up to two secondary labels, and flag off-topic noise”*, used to lift domain-label precision from 62% to 78% via a multi-agent verification pass.